

Refining Agent Behavior

Prompts, Intent, and Testing

A practical guide for developers and AI trainers building conversational agents that are consistent, trustworthy, and ready for real users.



System Prompt vs. User Message

Both are text sent to the model — but they carry different authority and serve different purposes.

System Prompt

- **Written by:** Developer or trainer
- **Sent:** Once, at the start of every turn
- **Controls:** Persona, scope, tone, rules
- **Priority:** Higher — overrides conflicting user intent

User Message

- **Written by:** End user
- **Sent:** Per turn, from the user
- **Controls:** The actual request or question
- **Priority:** Lower — processed within system boundaries

❏ Think of the system prompt as **standing orders**. The user message is the request that arrives within those orders. If the two conflict, the system prompt wins.

Prompt Patterns

A **prompt pattern** is a reusable structure for shaping how the agent handles a specific type of conversation. Patterns don't constrain the agent — they make it **consistent**. Consistent agents feel trustworthy. Inconsistent agents feel broken.

1

Greeting

Acknowledge the user warmly and signal what the agent can do.

"Hello! I am here to help you with your order, products, and returns."

2

Clarifying Question

When the request is ambiguous, ask before acting.

"Could you share your order number so I can check the status?"

3

Empathy Acknowledgment

When the user is frustrated, acknowledge before solving.

"I understand that is frustrating. Let me look into this for you."

4

Closing

Signal the end of a task and invite the next question.

"Is there anything else I can help you with today?"

Intent Modeling

An **intent** is the user's goal — the action or information they need from the agent. Users rarely state their intent clearly, so the model must infer it.

What the user says	Likely intent
"It hasn't arrived yet"	Track order
"This is broken"	Raise complaint or request return
"What do you recommend?"	Product guidance
"How do I return this?"	Return process
"Hi, I need help"	Triage or greeting

- ❶ Intent modeling does not require code at this stage. It lives in the instruction — well-written prompts handle routing implicitly. If no intent matches, the agent applies the fallback rule.

Prompts for Multiple Intents

The key is **structure** — **not length**. A long, unstructured prompt is harder for the model to follow than a short, well-structured one. Think of each intent block as a mini-contract the model honours when the matching intent is detected.

A Pattern That Scales

You **are** [persona].

You help users **with**:

[list **of** supported topics].

You do **not** help **with**:

[**out-of-scope** topics].

For each intent, follow these rules:

- **Order** status: ask **for order** number, **then** report what you know.
- **Returns**: confirm eligibility, **then** explain the process.
- Product queries: answer **from** available product info only.
- Complaints: acknowledge, empathize, **then** offer resolution steps.
- **Unknown**: say you do **not** have that information. Do **not** guess.

What Breaks Without Structure

- The model picks the nearest plausible intent instead of the correct one
- Edge cases fall through and produce hallucinated answers
- Tone becomes inconsistent between intents

What Works With Structure

- Clear scope boundaries keep the agent on task
- Explicit fallback rules prevent guessing
- Consistent tone across all intent types

ADK Web — Observing Agent Behavior

ADK Web is a local browser interface for testing and debugging agents during development. It is your **prompt workshop** — use it to validate behavior before distributing to real users.

What It Shows Per Turn

- The agent's full reply
- Session state — all key-value pairs currently held
- Any tool calls made — inputs sent and outputs received
- Step-by-step reasoning traces where available
- Logs per message, per tool, and per state change

The Refinement Loop

01

Send a test message

02

Read the reply and the trace

03

Identify what is missing, inconsistent, or wrong

04

Update the instruction

05

Repeat until the reply matches expectations

Testing Agents — It's Different

Testing an agent is fundamentally different from testing a function that always returns the same output for the same input. The model's output is **probabilistic** — you are testing for intent, tone, scope, and correctness, not exact text matches.

In-Scope

Does the agent answer correctly for a valid, expected request?

Out-of-Scope

Does the agent decline politely instead of guessing or hallucinating?

Edge Case

Does the agent handle ambiguous, incomplete, or off-topic input gracefully?

Example Test Case

✓ **Input:** "I want to return my laptop."
Expected: Agent confirms return policy, asks for order number or purchase date.

✗ **Not expected:** Agent invents a return approval or provides a fake reference number.

A test case for an agent is a **behavioral contract**, not a string comparison.

Designing Tests That Expose Weaknesses

The goal is not to test what works — it is to find what breaks. If a scenario exposes a weak reply, **fix the prompt, not the test.**

Ambiguous Input

"I have a problem." — Does the agent ask what kind, or guess?

Missing Context

"Where is it?" — Does the agent ask for clarification, or assume?

Boundary Push

"Can you help me write an email to my manager?" — Does the agent stay in scope or drift?

Emotional Escalation

"This is ridiculous. I've been waiting three weeks." — Does the agent acknowledge frustration first?

Unknown Data Request

"What is today's price for the Pro X laptop?" — Does the agent admit it cannot access live data?

Contradictory Request

"I want to return it and keep it." — Does the agent handle the contradiction gracefully?

Making Prompt Changes Traceable

A prompt is code. It should be versioned, reviewed, and tested like code. **Prompt engineering without version control is debugging without logs** — you will not remember what changed or why it mattered.

What to Avoid

- Hardcoding the instruction as a plain string inside the agent definition
- Editing instructions without a record of what changed and why
- Testing a new instruction without comparing against the previous version

A Simple Traceability Structure

```
src/agents/  
  support_agent.py  ← agent definition  
instructions/  
  support_v1.txt    ← original instruction  
  support_v2.txt    ← refined instruction  
  support_v3.txt    ← current instruction  
tests/  
  test_support_agent.py ← test cases
```

Each version should record: **what** changed, **why** it changed, and whether previous tests still pass.

Good Instructions vs. Fragile Ones

A **good instruction** holds up across all valid inputs, edge cases, and users. A **fragile instruction** works for the examples you wrote it for — and fails on everything else.

Signs of a Fragile Instruction

Symptom	Likely Cause
Different answers for similar questions	Intent detection not specific enough
Drifts off topic under pressure	Scope rules too vague
Different tones per intent	Tone guidance missing or inconsistent
Invents data when it has none	No explicit "say I don't know" rule
Confused by multi-part questions	No instruction for compound inputs

Signs of a Good Instruction

- Names every supported intent and what to do for each
- Names what is out of scope and how to decline
- Gives a clear fallback rule for unknown inputs
- Specifies tone consistently throughout
- Runs without changes across all test scenarios

⚠️ Fragile instructions pass your own test cases and fail on real users. Write instructions for users, not for yourself.

Knowing When Your Agent Is Ready

A single-agent system is ready for the next phase when it passes a minimum quality bar across all test dimensions. This is a **development completeness check** — the foundation that the next capabilities build on.



Intent Coverage

All supported intents return correct, in-scope replies.



Scope Enforcement

Out-of-scope queries are declined politely with no hallucination.



Clarification Behavior

Ambiguous queries trigger a clarifying question rather than a guess.



Empathy Handling

Emotional messages receive an empathetic response before a solution.



Honest Fallback

Unknown data requests return an honest "I don't know" — never invented facts.



Versioned & Documented

The instruction is versioned and all test cases are documented.

What to Explore Next

If the concepts here felt solid, these questions point toward the concrete techniques that come next. Each one maps to a real capability you will build.



Tools & Live Data

How do you connect an agent to live data — what does it mean to give an agent a "tool"?



Cross-Session Memory

When the agent needs to remember something across conversations — not just within one — how does that work?



Concurrent Users

When two different users run the same agent simultaneously — how is their context kept separate?



Automated Test Suites

How do you make a test suite that runs automatically rather than manually?



Multi-Agent Architecture

When an agent grows complex enough that one instruction cannot handle everything — what is the next step?